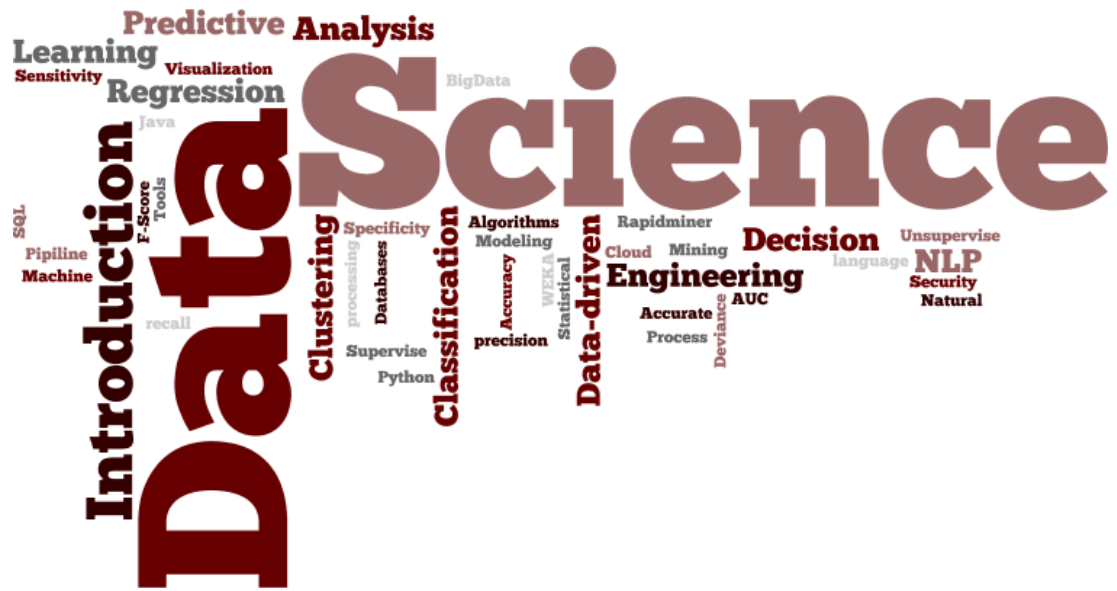




5. วิศวกรรมคุณลักษณะและการคัดเลือกแบบจำลอง

Feature Engineering and Model Selection

ดำรงฤทธิ์ เศรษฐศิริโชค

5. การคัดเลือกแบบจำลองและวิศวกรรมคุณลักษณะ

5.1 วิศวกรรมคุณลักษณะ (Feature Engineering)

5.2 การคัดเลือกแบบจำลอง

5.3 Validation Curves และ Learning Curves

5.4 Grid Search

5.5 กรณีตัวอย่าง การจำแนกภาพดิจิทัล



5.1 วิศวกรรมคุณลักษณะ (Feature Engineering)

- เป็นขั้นตอนอันดับแรกๆ ที่สำคัญมากในการเตรียมข้อมูล
- ข้อมูลจริงไม่ได้มาในรูปแบบที่พร้อมใช้ [n_samples, n_features]
- ต้องเข้าใจคุณสมบัติ (properties) หรือคุณลักษณะ (features) ของข้อมูลก่อน
- คัดเลือกคุณลักษณะจากองค์ความรู้ด้านที่เกี่ยวข้อง
- การแยกคุณลักษณะ (splitting features), การรวมคุณลักษณะ (combining features)
- การสร้างคุณลักษณะขึ้นมาใหม่ (creating new features)
- มุ่งเน้นเพื่อผลลัพธ์แบบจำลองที่ดีขึ้น แทนการเพิ่มความซับซ้อนของแบบจำลอง

Features = Attributes

ลักษณะประจำ (Attribute)

- ลักษณะเฉพาะเพื่ออธิบายถึงสิ่งต่างๆ

Attribute Type		Description	Examples	Operations
Categorical (Qualitative)	Nominal	The values of a nominal attribute are just different names; i.e., nominal values provide only enough information to distinguish one object from another. (=, ≠)	zip codes, employee ID numbers, eye color, gender	mode, entropy, contingency correlation, χ^2 test
	Ordinal	The values of an ordinal attribute provide enough information to order objects. (<, >)	hardness of minerals, { <i>good</i> , <i>better</i> , <i>best</i> }, grades, street numbers	median, percentiles, rank correlation, run tests, sign tests
Numeric (Quantitative)	Interval	For interval attributes, the differences between values are meaningful, i.e., a unit of measurement exists. (+, -)	calendar dates, temperature in Celsius or Fahrenheit	mean, standard deviation, Pearson's correlation, <i>t</i> and <i>F</i> tests
	Ratio	For ratio variables, both differences and ratios are meaningful. (×, /)	temperature in Kelvin, monetary quantities, counts, age, mass, length, electrical current	geometric mean, harmonic mean, percent variation

Categorical Features

```
In [1]: data = [  
    {'price': 850000, 'rooms': 4, 'neighborhood': 'Queen Anne'},  
    {'price': 700000, 'rooms': 3, 'neighborhood': 'Fremont'},  
    {'price': 650000, 'rooms': 3, 'neighborhood': 'Wallingford'},  
    {'price': 600000, 'rooms': 2, 'neighborhood': 'Fremont'}  
]
```

ต้องทำ OneHot Encoding

- ก่อนนำไปใช้งานต้องปรับให้อยู่ในรูปตาราง หรือ Dataframe ใน Python

DictVectorizer

```
In [3]: from sklearn.feature_extraction import DictVectorizer  
vec = DictVectorizer(sparse=False, dtype=int)  
vec.fit_transform(data)
```

```
Out[3]: array([[ 0,  1,  0, 850000,  4],  
               [ 1,  0,  0, 700000,  3],  
               [ 0,  0,  1, 650000,  3],  
               [ 1,  0,  0, 600000,  2]])
```


'neighborhood'

```
In [4]: vec.get_feature_names_out()
```

```
Out[4]: array(['neighborhood=Fremont', 'neighborhood=Queen Anne',  
               'neighborhood=Wallingford', 'price', 'rooms'], dtype=object)
```

Sparse Matrix

- ควรใช้ sparse matrix กับข้อมูลที่มี category จำนวนมาก ซึ่ง matrix จะมีค่าส่วนใหญ่เป็น 0
- ทำให้การทำงานเร็วขึ้น ไม่เปลือง memory

```
In [5]: vec = DictVectorizer(sparse=True, dtype=int)
        vec.fit_transform(data)
Out[5]: <4x5 sparse matrix of type '<class 'numpy.int64'>'
        with 12 stored elements in Compressed Sparse Row format>
```

ศึกษาเพิ่มเติม

```
sklearn.preprocessing.OneHotEncoder
sklearn.feature_extraction.FeatureHasher
```

Text Features

- ต้องแปลงให้อยู่ในรูปตัวเลข
 - นับความถี่ของคำศัพท์แต่ละคำ → scale ของความถี่จะต่างกันเยอะ
 - Term Frequency Inverse Document Frequency: TF-IDF
เพิ่มปัจจัย IDF เพื่อเทียบกับ Document ทั้งหมด

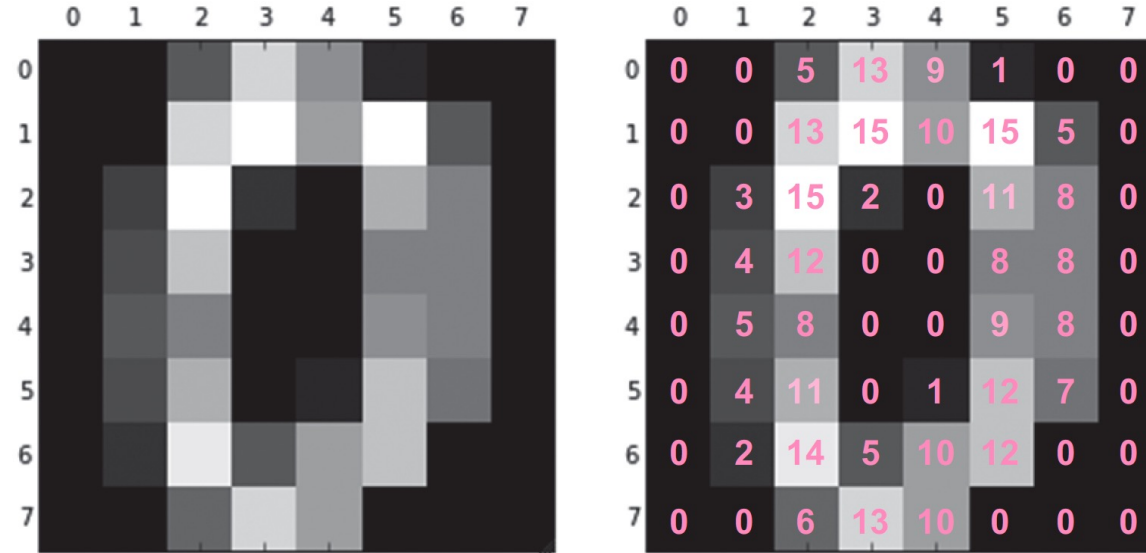
```
In [6]: sample = ['problem of evil',  
                  'evil queen',  
                  'horizon problem']
```



evil	horizon	of	problem	queen
1	0	1	1	0
1	0	0	0	1
0	1	0	1	0

	evil	horizon	of	problem	queen
TF-IDF	0.517856	0.000000	0.680919	0.517856	0.000000
	0.605349	0.000000	0.000000	0.000000	0.795961
	0.000000	0.795961	0.000000	0.605349	0.000000

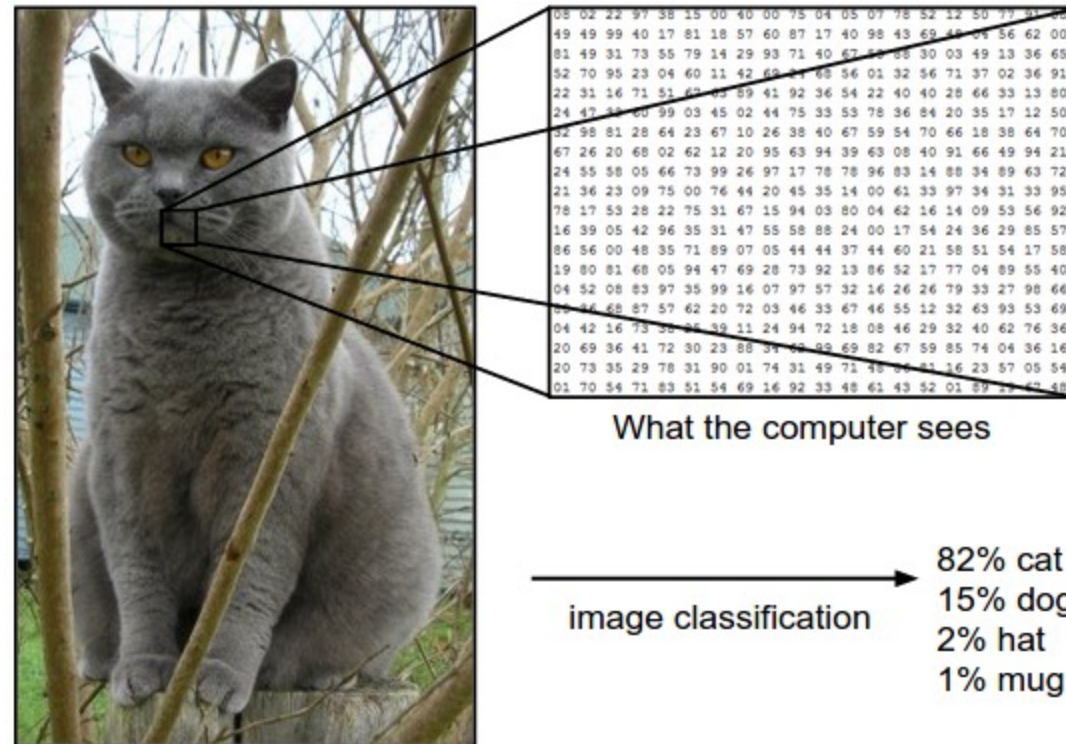
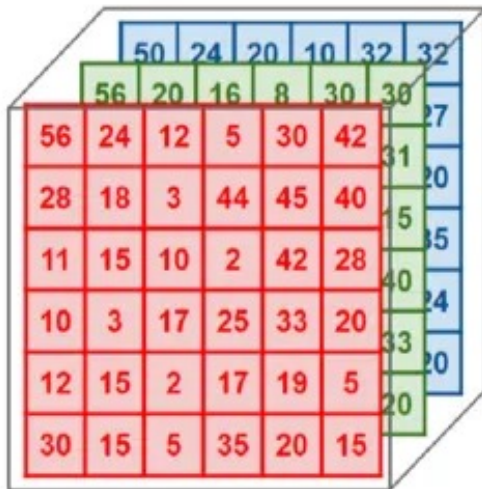
Image Features



```
from sklearn.datasets import load_digits
import pylab as pl
digits = load_digits()
pl.gray()
pl.matshow(digits.images[0])
pl.show()
digits.images[0]
```

```
array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.],
       [ 0.,  0., 13., 15., 10., 15.,  5.,  0.],
       [ 0.,  3., 15.,  2.,  0., 11.,  8.,  0.],
       [ 0.,  4., 12.,  0.,  0.,  8.,  8.,  0.],
       [ 0.,  5.,  8.,  0.,  0.,  9.,  8.,  0.],
       [ 0.,  4., 11.,  0.,  1., 12.,  7.,  0.],
       [ 0.,  2., 14.,  5., 10., 12.,  0.,  0.],
       [ 0.,  0.,  6., 13., 10.,  0.,  0.,  0.]])
```

ภาพสี่



<https://www.kdnuggets.com/2019/11/deep-learning-image-classification-less-data.html>

Derived Features

- บางคุณลักษณะสามารถสร้างขึ้นจากคุณลักษณะที่มีอยู่ได้
- รวมถึง Discretization Binarization
- Variable transformation ตัวอย่างเช่น
 - Simple Functions

$$x = \log(x)$$

- Normalization: ปรับข้อมูลให้อยู่ในช่วงที่กำหนด

$$x = (x - \min(x)) / (\max(x) - \min(x))$$

- Standardization: ปรับค่าเฉลี่ยให้อยู่ที่ 0 ทำให้ค่าไม่ใหญ่เกินไป กระจายตัวสม่ำเสมอ (mean=0, SD=1)

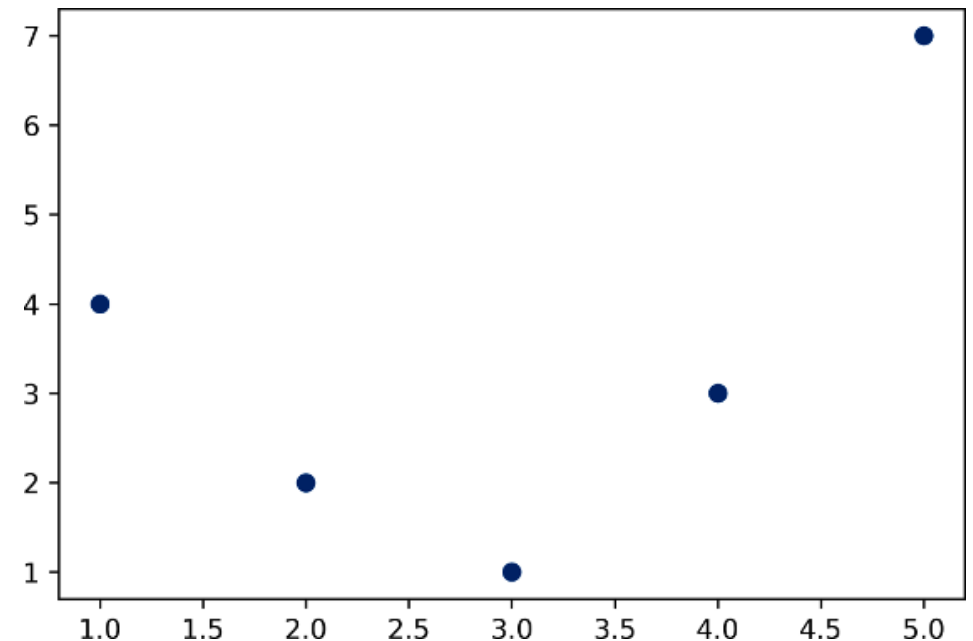
$$x' = \frac{(x - \bar{x})}{S_x}$$

Derived Features

- ตัวอย่าง
หากต้องการสร้าง Polynomial Regression
สามารถสร้างจากข้อมูลเข้าให้อยู่ในรูป Polynomial
- จากภาพหากใช้ Linear Regression จะไม่สามารถ
ได้แบบจำลองที่ประสิทธิภาพดีได้ เนื่องจาก x และ y
มีความสัมพันธ์แบบ non-linear

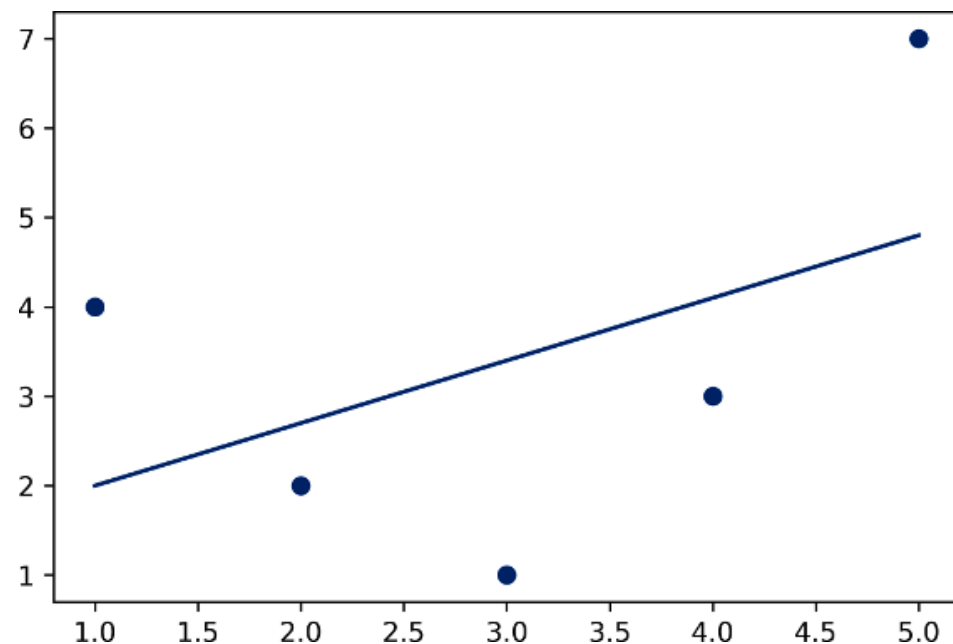
```
x = np.array([1, 2, 3, 4, 5])
```

```
y = np.array([4, 2, 1, 3, 7])
```



LinearRegression

```
In [11]: from sklearn.linear_model import LinearRegression
X = x[:, np.newaxis]
model = LinearRegression().fit(X, y)
yfit = model.predict(X)
plt.scatter(x, y)
plt.plot(x, yfit);
```

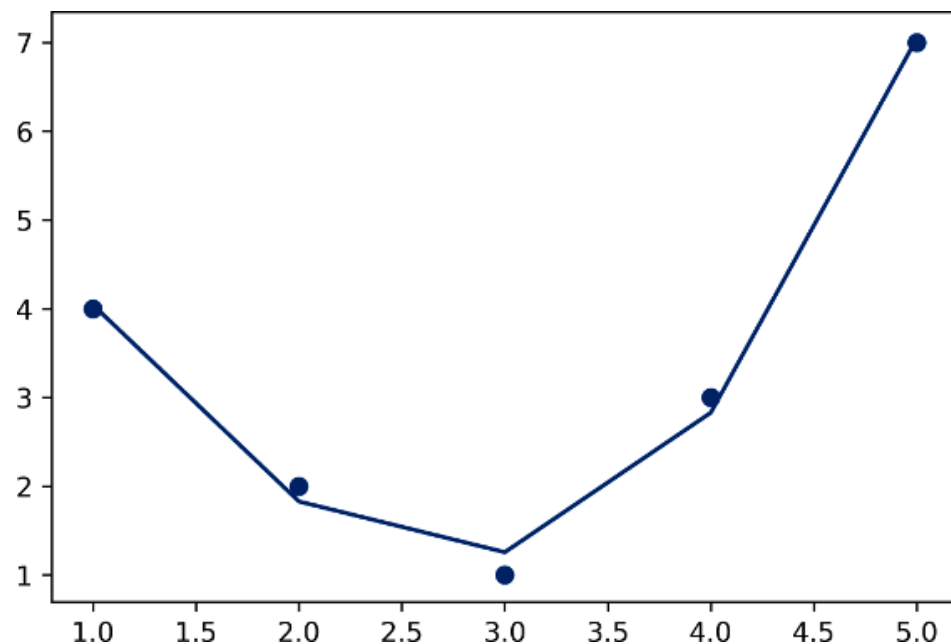


PolynomialFeatures

```
In [12]: from sklearn.preprocessing import PolynomialFeatures  
poly = PolynomialFeatures(degree=3, include_bias=False)  
X2 = poly.fit_transform(X)  
print(X2)
```

```
Out[12]: [[ 1.  1.  1.]  
          [ 2.  4.  8.]  
          [ 3.  9. 27.]  
          [ 4. 16. 64.]  
          [ 5. 25. 125.]]
```

```
In [13]: model = LinearRegression().fit(X2, y)  
yfit = model.predict(X2)  
plt.scatter(x, y)  
plt.plot(x, yfit);
```



Imputation of Missing Data

- จะทำอย่างไรกับ NaN
- แบบจำลองรองรับค่า missing บางแบบจำลองเท่านั้น
- ต้องทำการแทนค่าด้วยค่าที่เหมาะสม
- วิธีปรับเปลี่ยนไปตามแอปพลิเคชัน
- พื้นฐานจะแทนด้วย 0, mean, median, most frequent

```
from numpy import nan
X = np.array([[ nan,    0,    3 ],
               [  3,    7,    9 ],
               [  3,    5,    2 ],
               [  4,   nan,    6 ],
               [  8,    8,    1 ]])

y = np.array([14, 16, -1, 8, -5])
```

SimpleImputer

```
In [15]: from sklearn.impute import SimpleImputer
         imp = SimpleImputer(strategy='mean')
         X2 = imp.fit_transform(X)
         X2
```

```
Out[15]: array([[4.5, 0. , 3. ],
                [3. , 7. , 9. ],
                [3. , 5. , 2. ],
                [4. , 5. , 6. ],
                mean [8. , 8. , 1. ]])
```



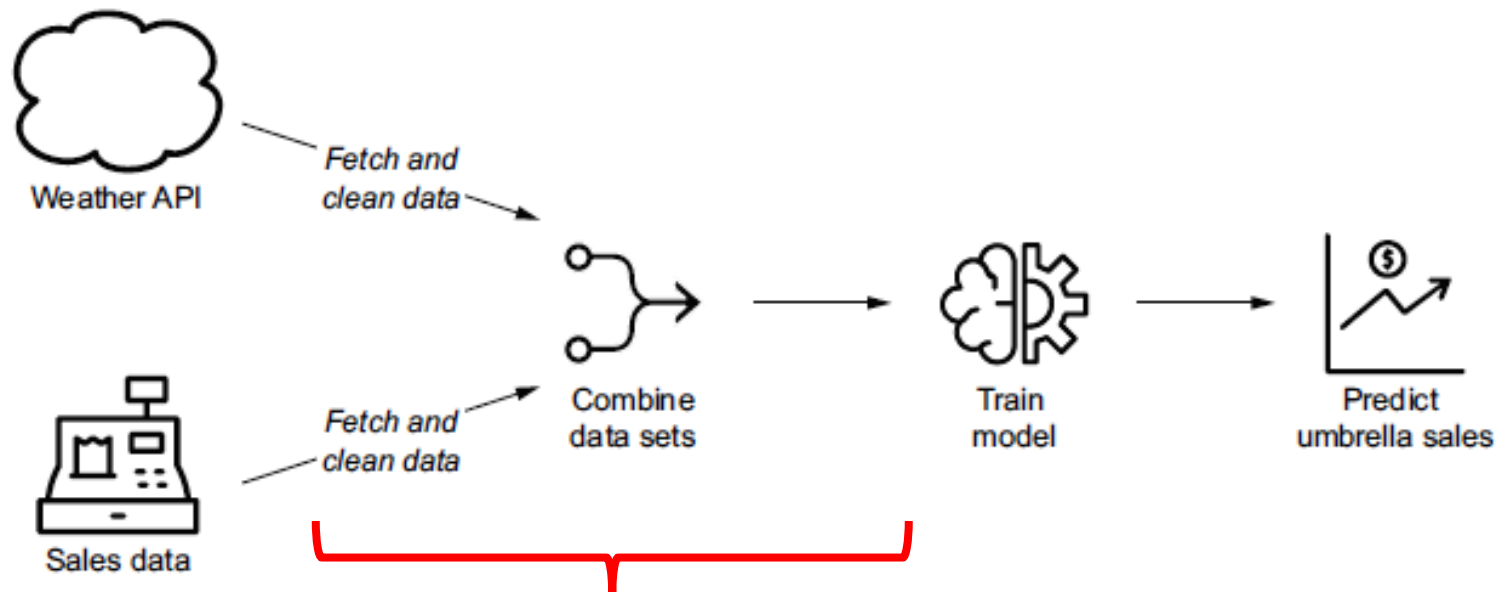
“mean”, “median”,
“most_frequent”, “constant”

```
In [16]: model = LinearRegression().fit(X2, y)
         model.predict(X2)
```

```
Out[16]: array([13.14869292, 14.3784627 , -1.15539732,
                10.96606197, -5.33782027])
```

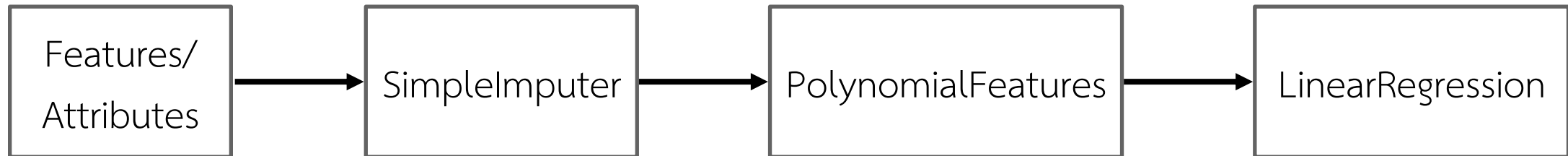
Feature Pipelines

- Data Pipeline – ลำดับการเชื่อมต่อของกระบวนการงาน



Feature Pipelines

Feature Pipelines



```
In [17]: from sklearn.pipeline import make_pipeline
         model = make_pipeline( SimpleImputer(strategy='mean'),
                                PolynomialFeatures(degree=2),
                                LinearRegression())
```

Q & A

5.2 การคัดเลือกแบบจำลอง

- จากการทดสอบโมเดล (Model Validation) 3 วิธี วิธี Cross-Validation เป็นวิธีที่สามารถประเมินประสิทธิภาพแบบจำลองได้ดี
 - Re-substitution
 - Hold-out
 - **K-folds Cross-Validation**
- ถ้าประเมินค่าประสิทธิภาพแบบจำลองได้ต่ำเกินไป ควรทำอย่างไรต่อไป ?
 - เพิ่มความซับซ้อนของแบบจำลอง → ลด bias
 - เพิ่มจำนวนตัวอย่างข้อมูล → ลด variance
 - เพิ่มคุณลักษณะข้อมูล

การฟิตเกิน (Overfitting)

Find the next number of the sequence

1, 3, 5, 7, ?

Correct solution

217341

because when

$$f(x) = \frac{18111}{2}x^4 - 90555x^3 + \frac{633885}{2}x^2 - 452773x + 217331$$

$$f(1)=1$$

$$f(2)=3$$

$$f(3)=5$$

$$f(4)=7$$

$$f(5)=217341$$

much solution

wow very logic

such function

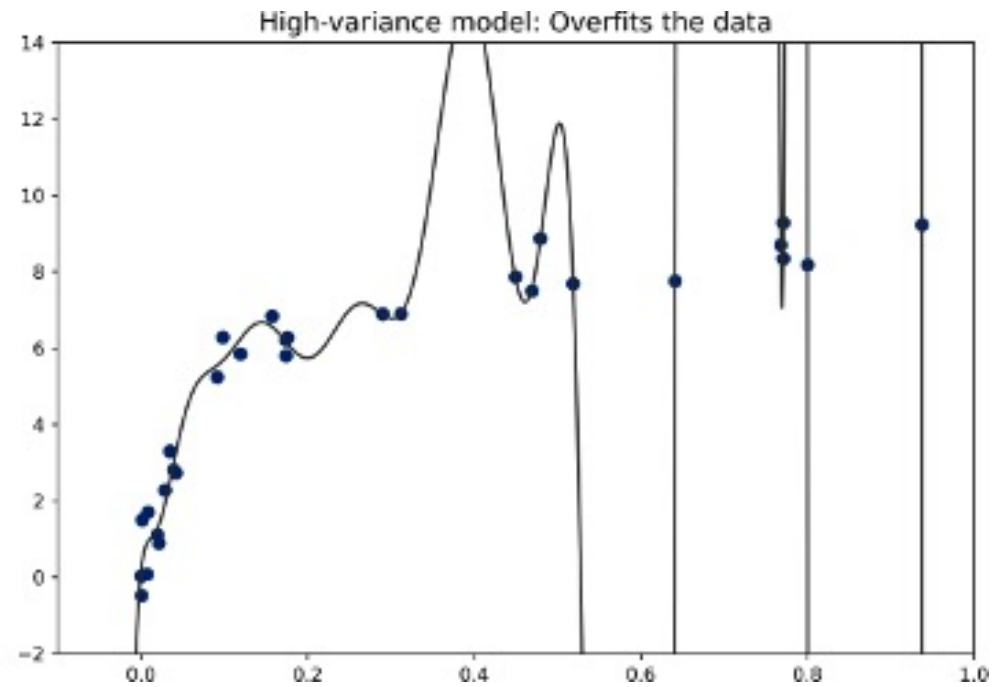
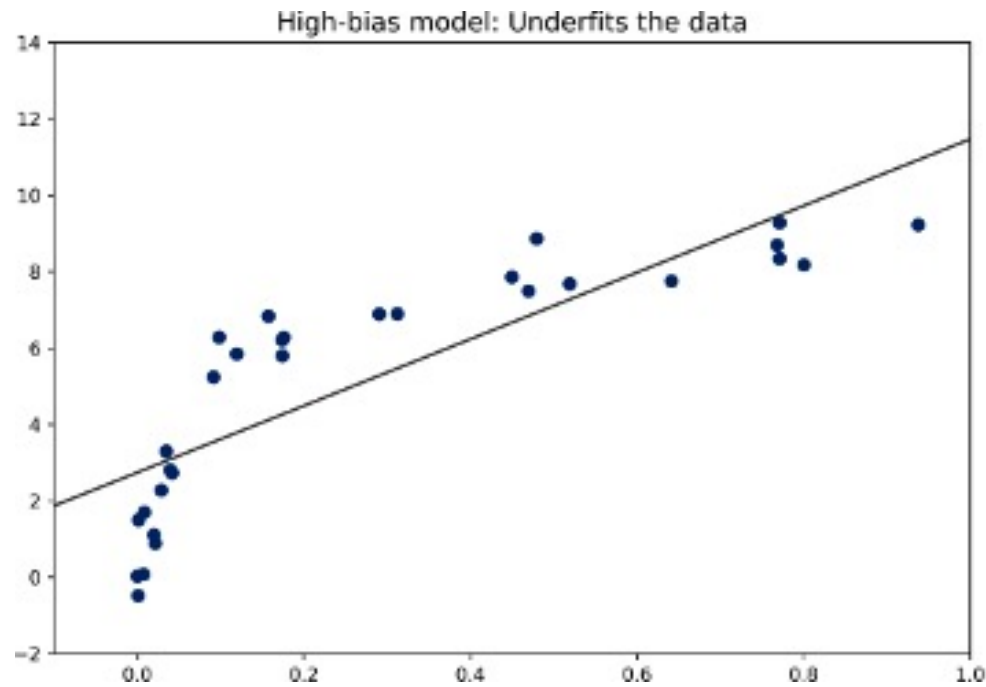
many maths

wow



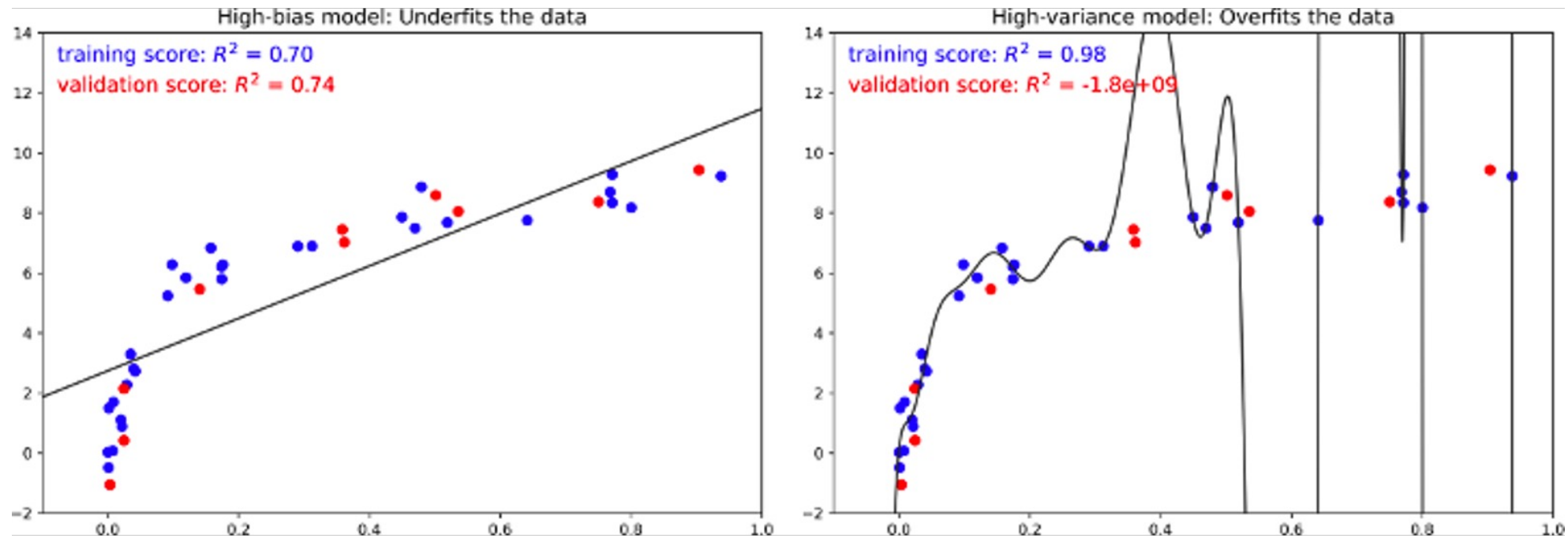
The Bias-Variance Trade-off

- แบบจำลองจะดีที่สุด คือจุดตัดสนใจระหว่าง bias และ variance



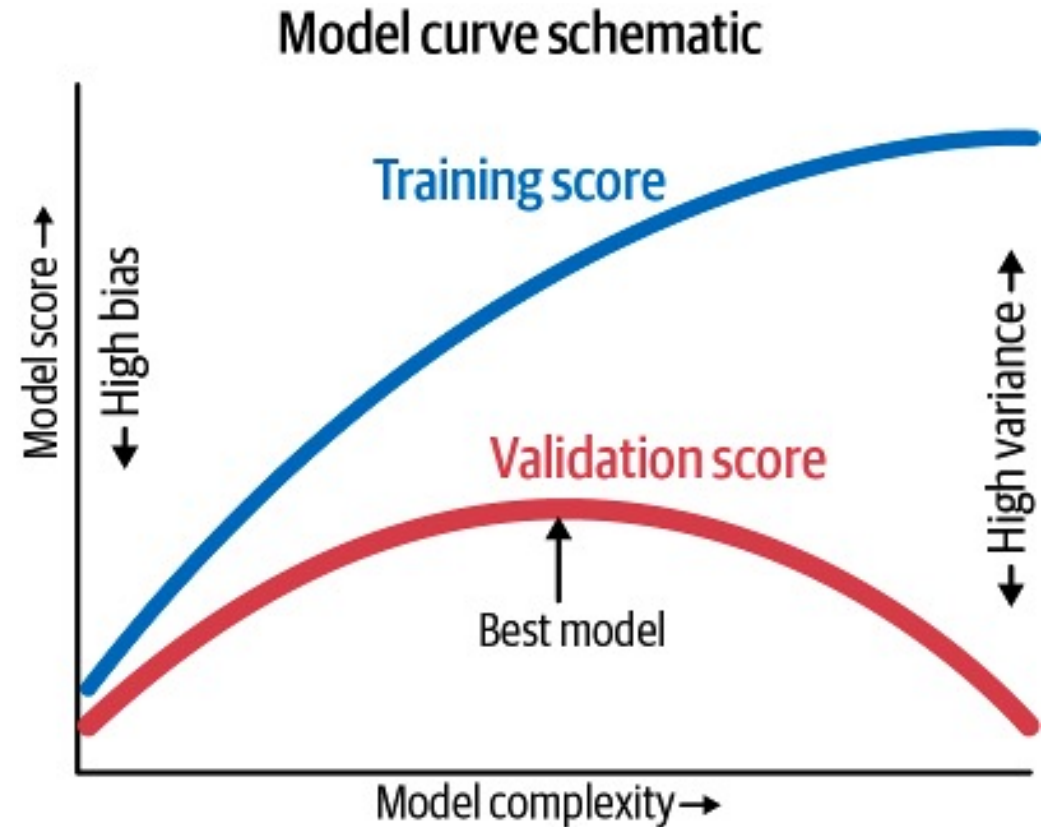
The Bias-Variance Trade-off

- R^2 = coefficient of determination ระบุว่าแบบจำลองมีความสัมพันธ์และกลมกลืนกับข้อมูลเพียงใด



The Bias-Variance Trade-off

- แบบจำลอง bias สูง
ประสิทธิภาพของ Training และ Validation จะใกล้เคียงกัน
- แบบจำลอง variance สูง
ประสิทธิภาพของ Validation จะแย่กว่า Training มาก
- แบบจำลองที่ดีที่สุด พิจารณา Validation Curve



5.3 Validation Curves

- เส้นโค้งผลลัพธ์ระหว่างประสิทธิภาพแบบจำลองจากข้อมูล training และ validation เทียบกับความซับซ้อนของแบบจำลอง
- ตัวอย่างการเรียนรู้ ใช้ข้อมูล polynomial

Degree-1: $y = ax + b$

Model parameters a, b

Degree-3: $y = ax^3 + bx^2 + cx + d$

Model parameters a, b, c, d

การสร้าง Polynomial Regression ใน sklearn



```
In [10]: from sklearn.preprocessing import PolynomialFeatures
         from sklearn.linear_model import LinearRegression
         from sklearn.pipeline import make_pipeline

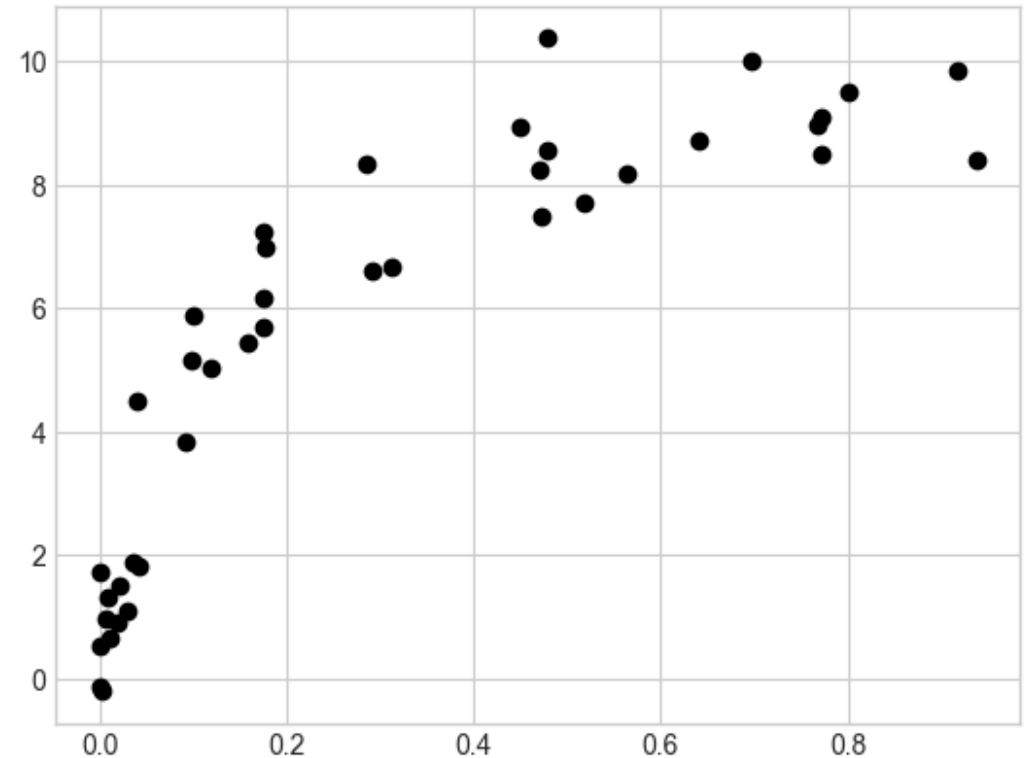
         def PolynomialRegression(degree=2, **kwargs):
             return make_pipeline(PolynomialFeatures(degree),
                                   LinearRegression(**kwargs))
```

การจำลองข้อมูลแบบมีสัญญาณรบกวน

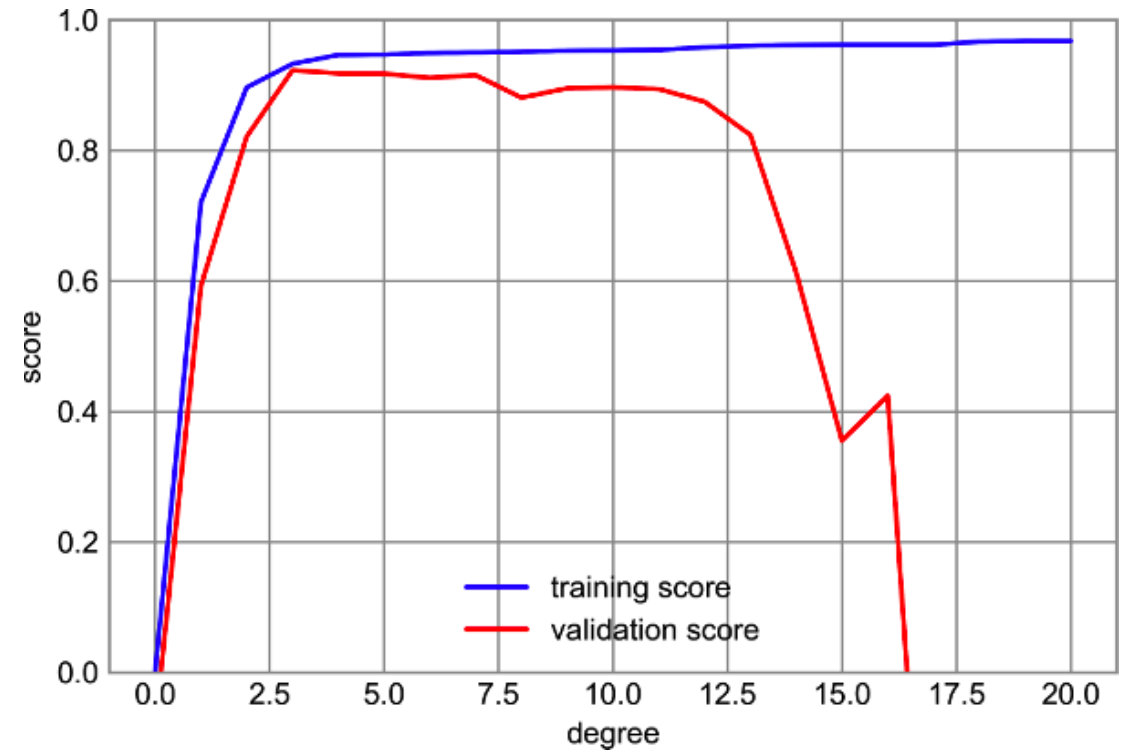
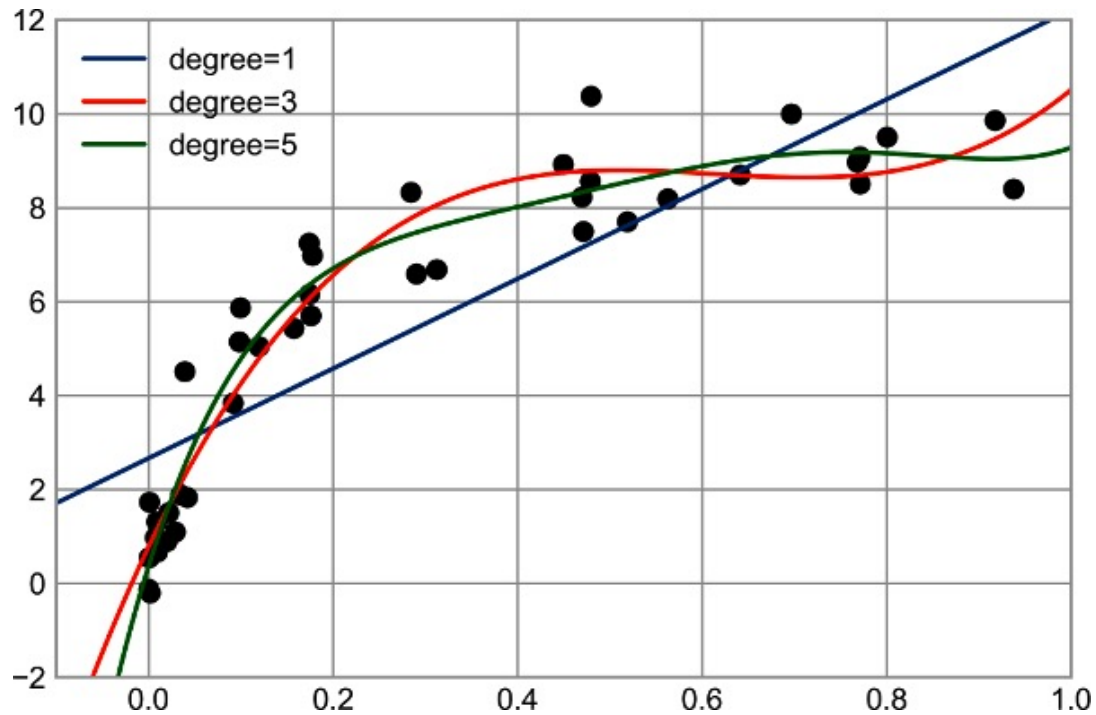
```
import numpy as np

def make_data(N, err=1.0, rseed=1):
    # randomly sample the data
    rng = np.random.RandomState(rseed)
    X = rng.rand(N, 1) ** 2
    y = 10 - 1. / (X.ravel() + 0.1)
    if err > 0:
        y += err * rng.randn(N)
    return X, y

X, y = make_data(40)
```

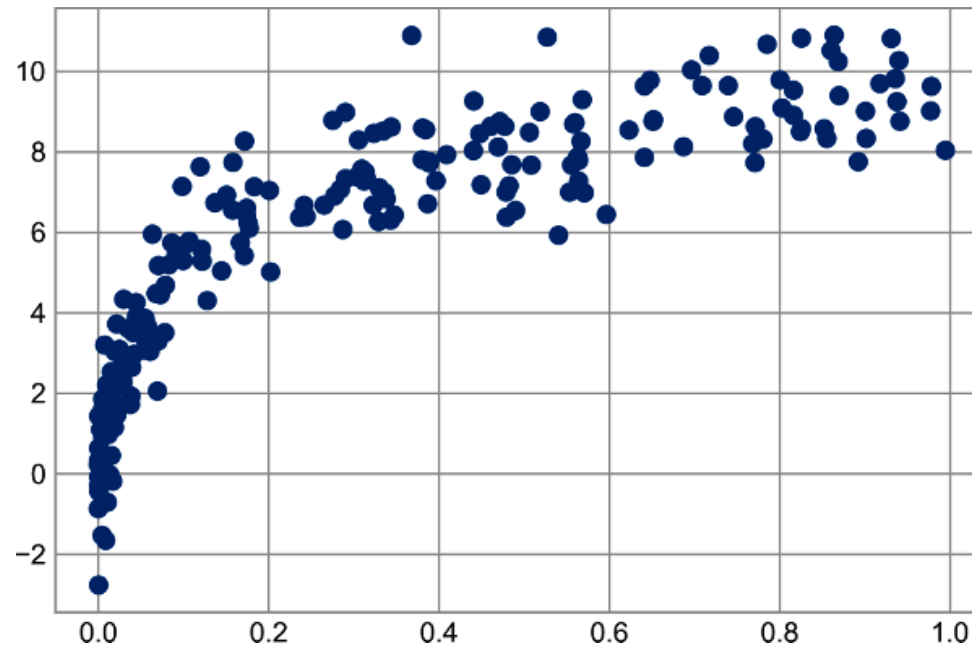


Validation Curves

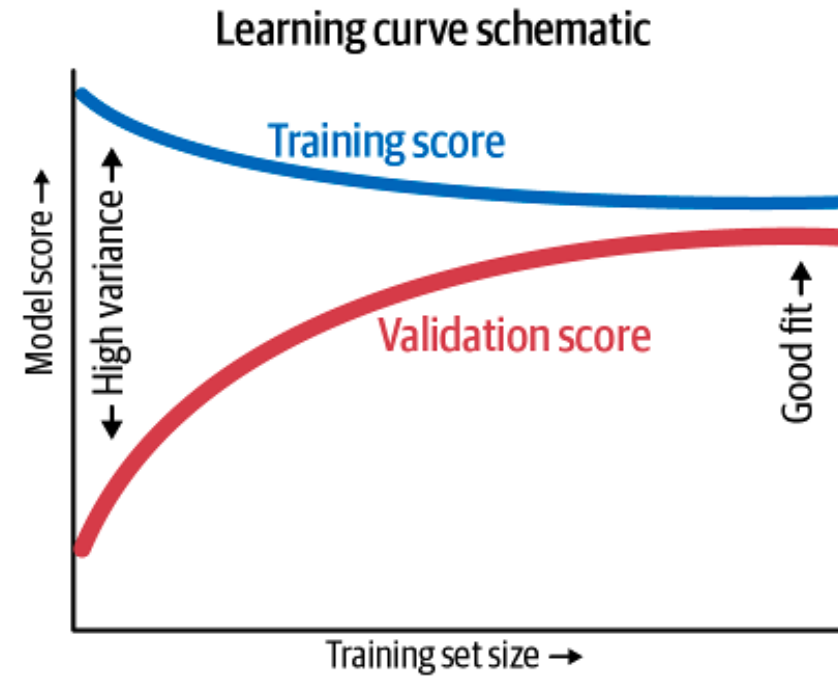
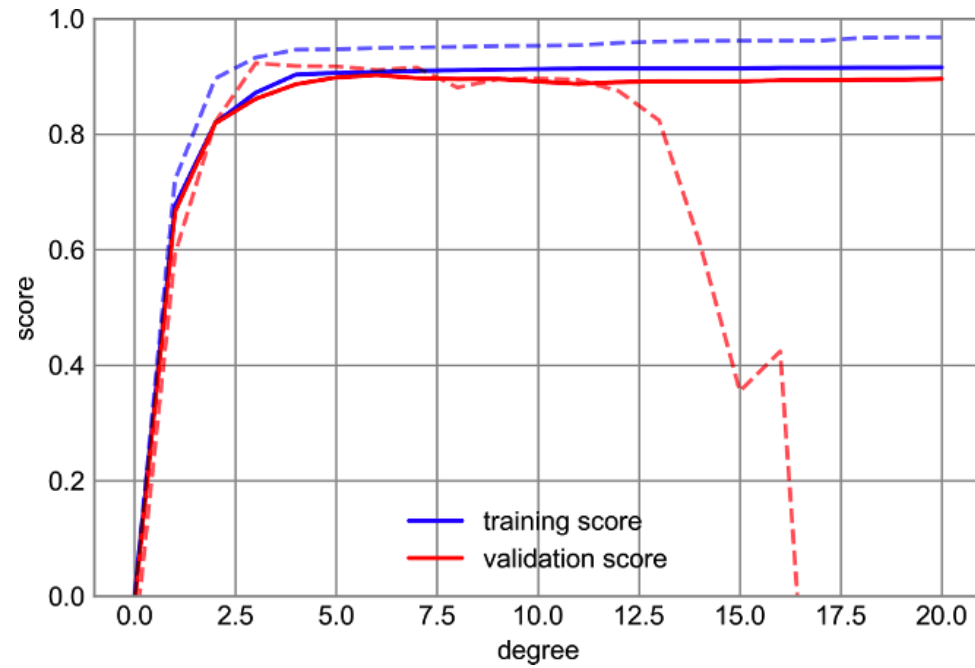


Learning Curves

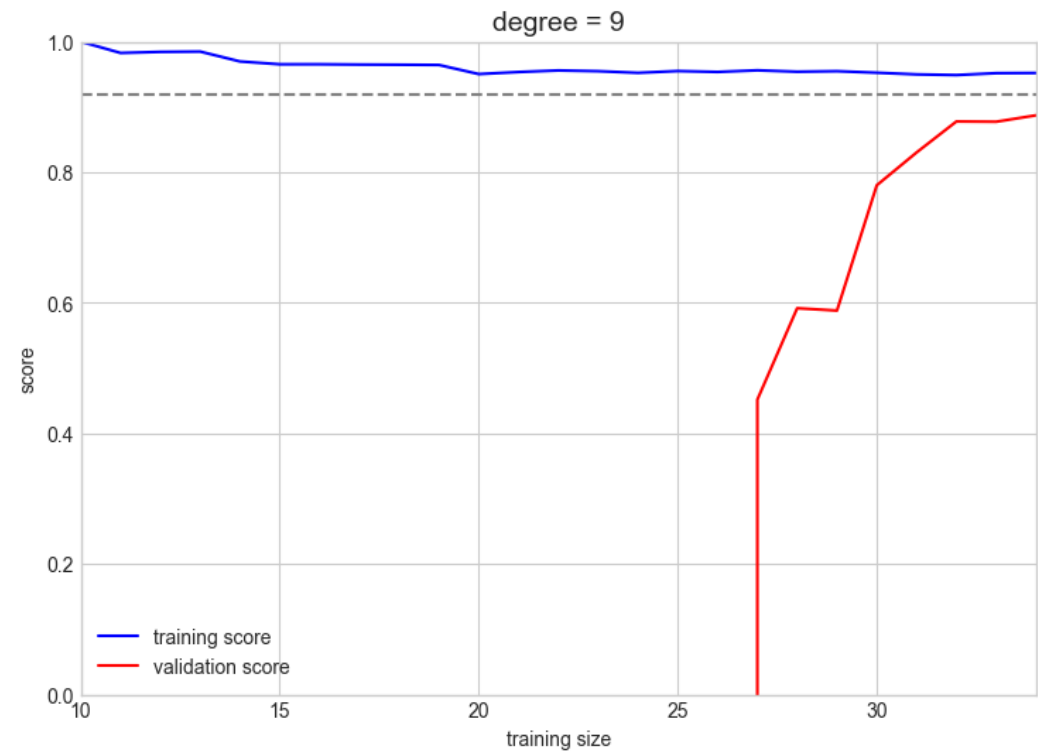
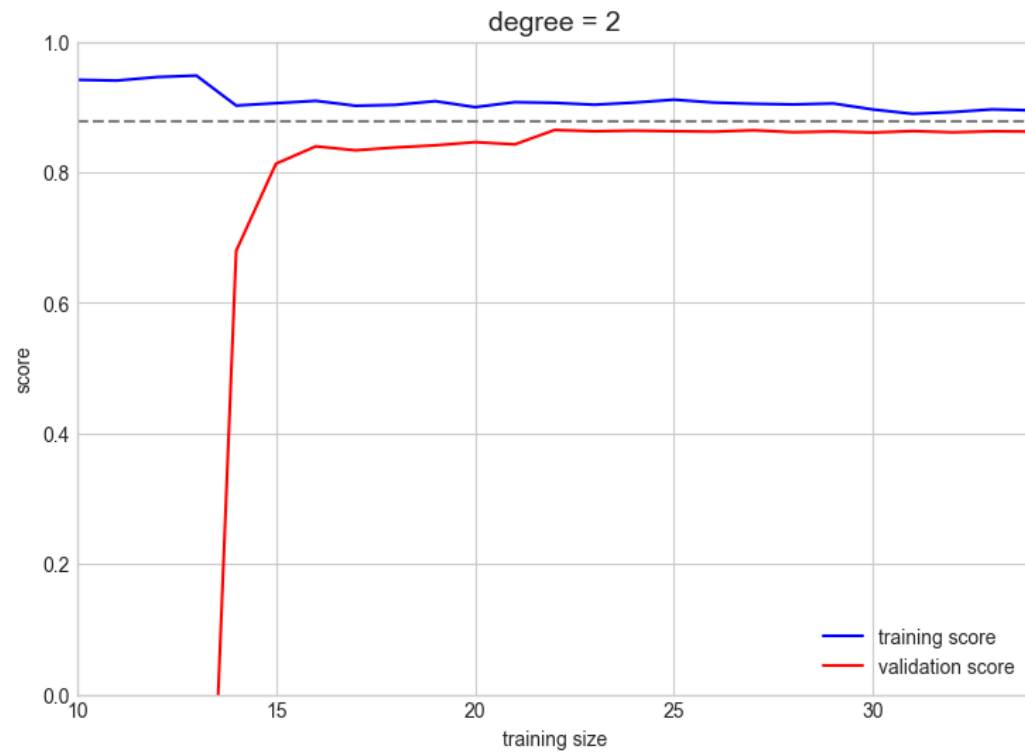
- เส้นโค้งผลลัพธ์ระหว่างประสิทธิภาพแบบจำลองจากข้อมูล training และ validation เทียบกับจำนวนตัวอย่างข้อมูล
- จำนวนตัวอย่างข้อมูลเพิ่มขึ้นมีโอกาสลดความแปรปรวนลง



Learning Curves



เปรียบเทียบ Learning Curves



- ถึงจุดหนึ่ง การเพิ่มจำนวนข้อมูลจะไม่ทำให้ประสิทธิภาพดีขึ้นอย่างมีนัยสำคัญ

5.4 Grid Search

- ในทางปฏิบัติ ไม่สามารถรู้คำตอบแบบจำลองที่ดีที่สุด
- พิจารณา Learning Curves หลายมิติจากจำนวนลักษณะประจำหลายตัว ความซับซ้อนสูง
- ต้องทำการ trial หาจุดที่ดีที่สุดโดยคำนึงถึง bias และ variance

```
In [18]: from sklearn.model_selection import GridSearchCV
         param_grid = {'polynomialfeatures__degree': np.arange(21),
                       'linearregression__fit_intercept': [True, False]}

         grid = GridSearchCV(PolynomialRegression(), param_grid, cv=7)
```

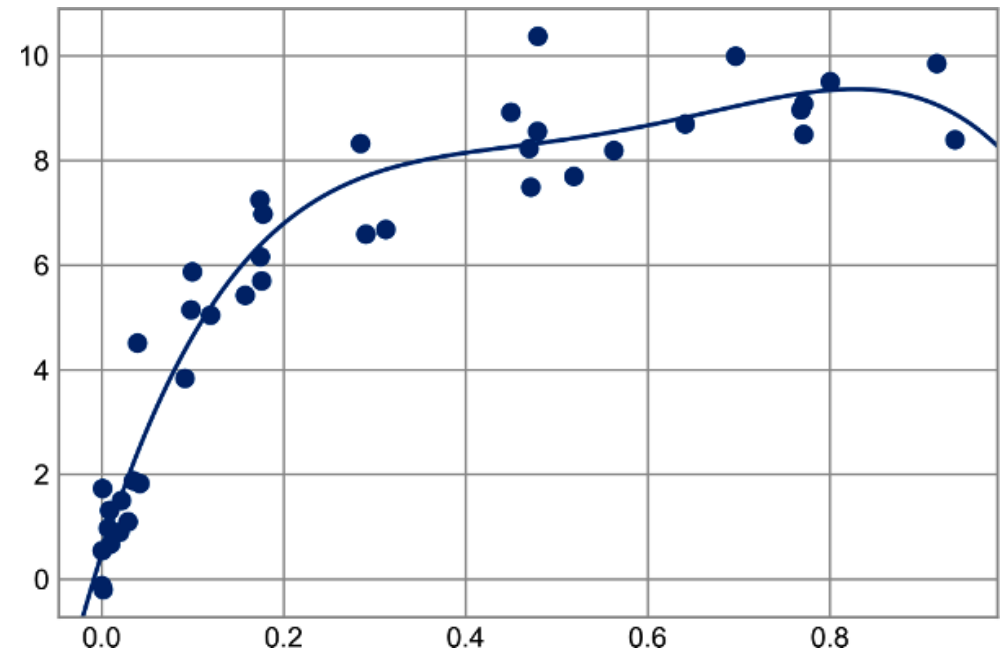
```
In [19]: grid.fit(X, y);
```

```
In [20]: grid.best_params_
Out[20]: {'linearregression__fit_intercept': False,
          'polynomialfeatures__degree': 4}
```

best_estimator_

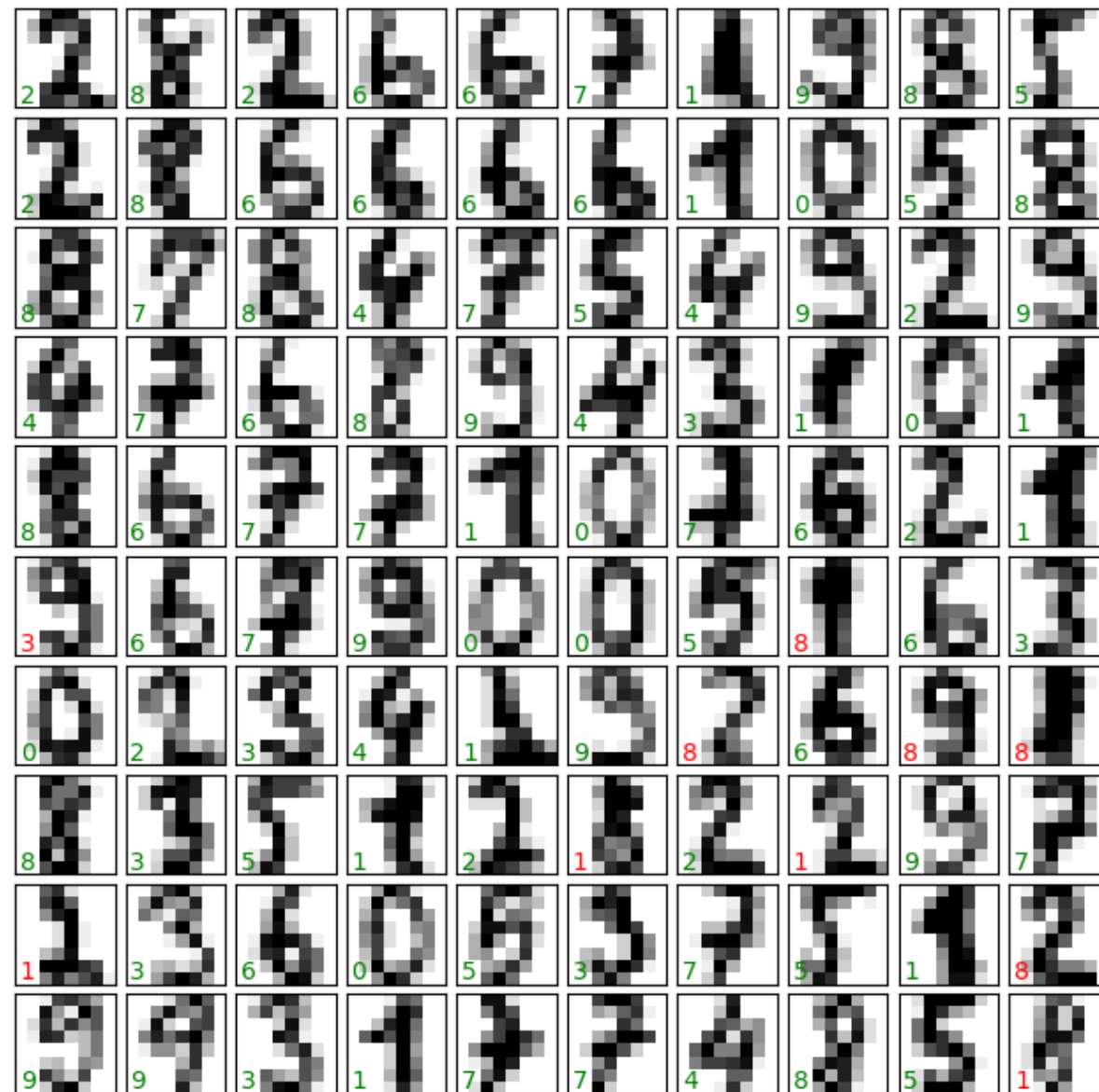
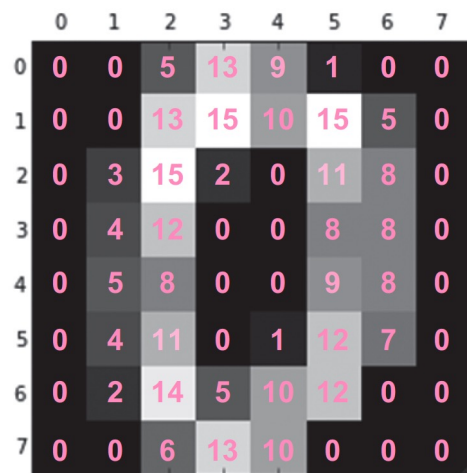
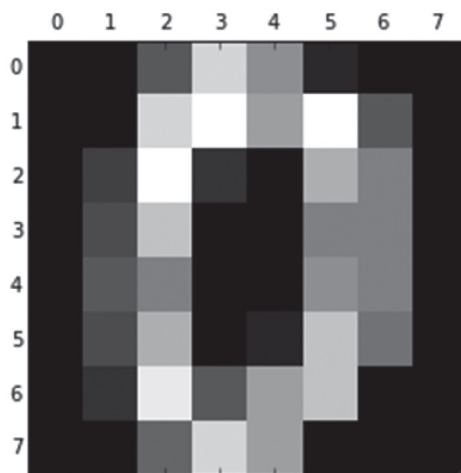
- เลือก model ที่ดีที่สุด นำออกมาใช้ได้ทันที

```
model = grid.best_estimator_  
plt.scatter(X.ravel(), y)  
lim = plt.axis()  
y_test = model.fit(X, y).predict(X_test)  
plt.plot(X_test.ravel(), y_test);  
plt.axis(lim);
```



5.5 กรณีสึกษา

การจำแนกภาพดิจิทัล

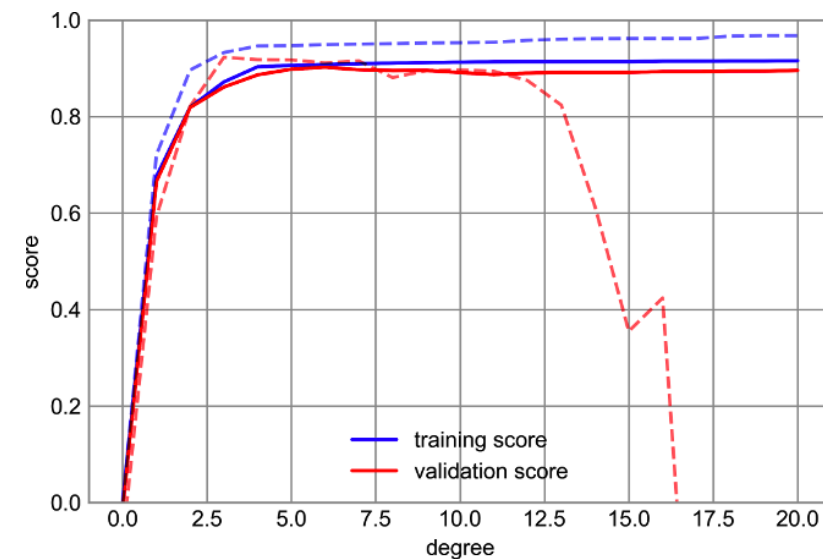
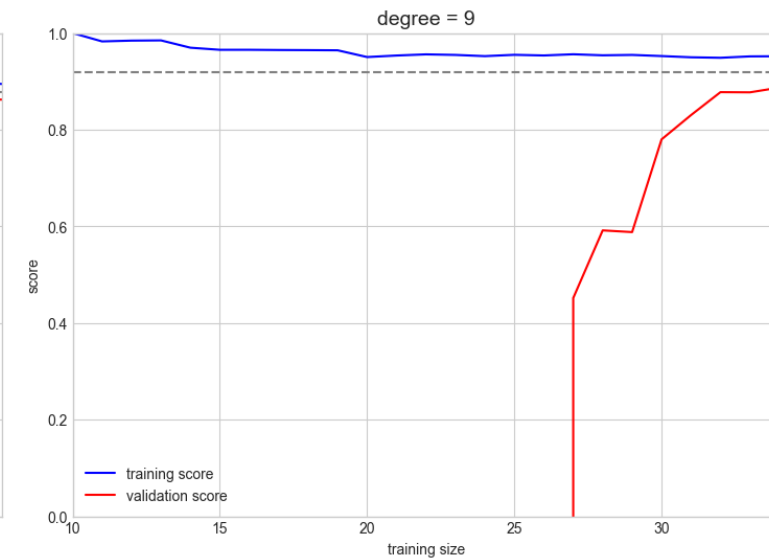
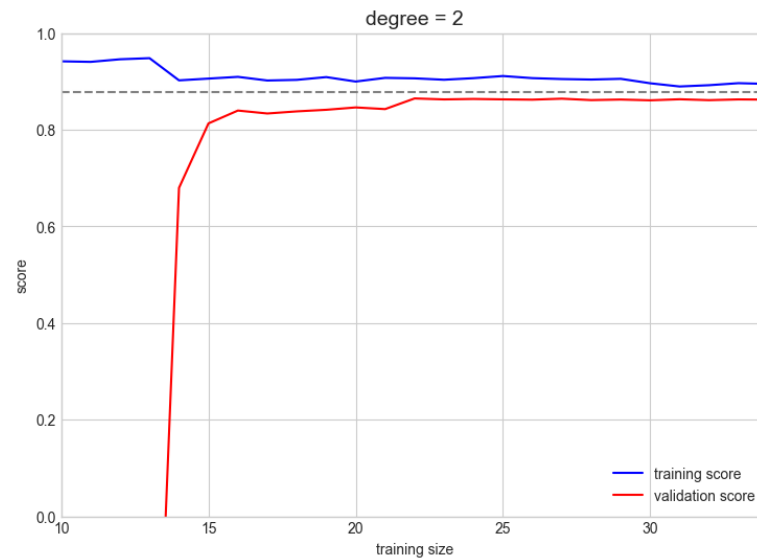


สรุป

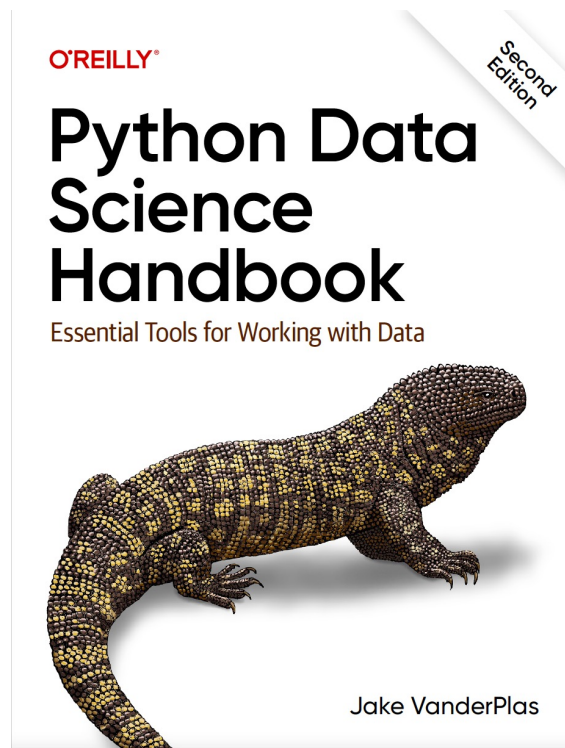
- วิศวกรรมคุณลักษณะ (Feature Engineering)
 - Data pipeline
 - Feature pipeline
 - ลักษณะประจำ (Attribute) = Feature
- การคัดเลือกแบบจำลอง
 - Bias-Variance Trade-off
 - Overfitting
 - Validation Curve และ Learning Curve
- Regression
 - Linear Regression มีข้อจำกัด
 - Polynomial Regression
- Classification
 - kNN (KNeighborsClassifier)
 - Naïve Bayes (GaussianNB)
- นำเข้าข้อมูล
 - File, Lib
 - API
 - Web Scraping

แบบฝึกหัดที่ #3

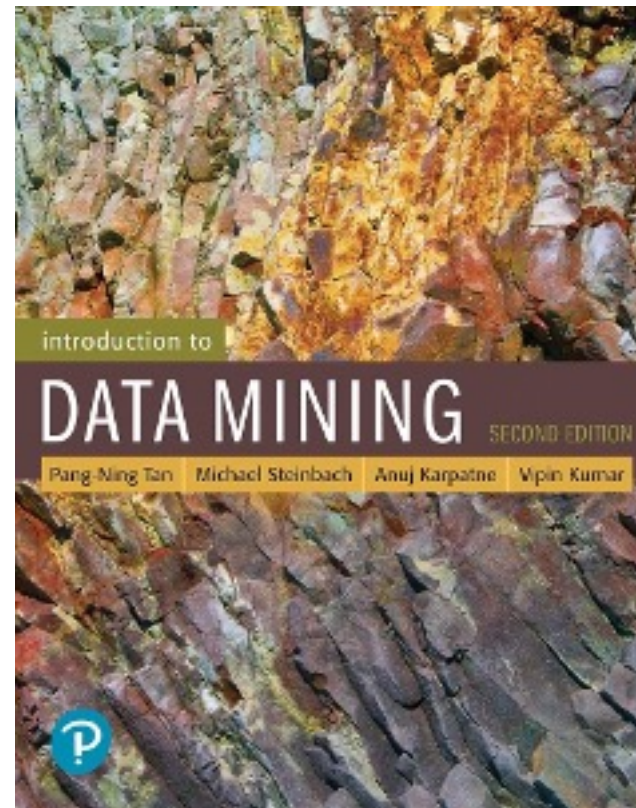
- วิเคราะห์ข้อมูล Real_estate.csv
- ทำความเข้าใจข้อมูล
- เตรียมข้อมูล
- ตรวจสอบ Correlation และ boxplot
- สร้างแบบจำลองทำนายค่า house price ด้วย **Polynomial Regression**
- พิจารณา Validation Curves ที่ **N=200 vs Full**
- เปรียบเทียบ **Learning Curves** กับประสิทธิภาพของแบบจำลอง หาแบบจำลองที่ดีที่สุดและอธิบายผลลัพธ์



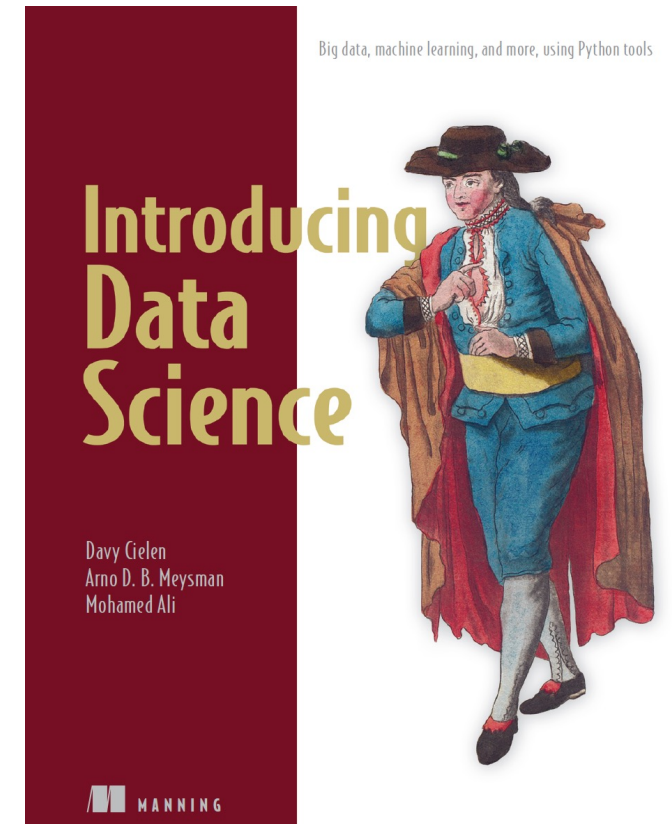
Ref



<https://github.com/jakevdp/PythonDataScienceHandbook>



<https://www-users.cse.umn.edu/~kumar001/dmbook/index.php>



Q & A